Tail Latency and Throughput Under Mixed Prompt Lengths in vLLM Serving

Shreyash Kondakindi
A69034537
University of California, San Diego
skondakindi@ucsd.edu

Susmit Singh
A69034300
University of California, San Diego
sus018@ucsd.edu

Abstract

Large language model (LLM) serving systems are frequently evaluated on average throughput, yet production deployments are governed by tail latency service-level objectives (SLOs) and must handle heterogeneous request streams. When short and long prompts arrive together, long-prefill requests can monopolize batch compute and memory bandwidth, a phenomenon known as head-of-line (HoL) blocking that disproportionately penalizes short requests and inflates the 99th-percentile (p99) latency. In this work, we present a reproducible performance study of the vLLM serving engine under controlled mixed prompt-length workloads. We construct short/long prompt mixes at varying ratios (90/10, 70/30, 50/50), introduce a dynamic context budgeting mechanism that prevents token-limit failures, and systematically sweep concurrency and two key server knobs: the per-batch token budget (`max-num-batched-tokens`) and admission capacity (`max-num-seqs`). Our measurements reveal that (i) mixing 30% long prompts into a short-only workload inflates p99 by over 50%, (ii) beyond the throughput saturation knee, additional concurrency yields negligible throughput gains while p99 grows by 3.5×, (iii) admission capacity acts primarily as a throughput lever and does not mitigate tail interference, and (iv) constraining the per-batch token budget combined with Pareto analysis over the two-dimensional configuration space enables SLO-aware operating-point selection.

1 Introduction

The deployment of large language models in interactive applications has grown rapidly over the past several years. Systems such as chatbots, code assistants, and retrieval-augmented generation pipelines serve millions of requests per day, and the user experience depends critically on response latency [Dean and Barroso, 2013]. While the research community has invested heavily in improving average inference throughput through techniques like continuous batching [Yu et al., 2022], paged key-value cache management [Kwon et al., 2023], and chunked prefill scheduling [Agrawal et al., 2024], production deployments are typically constrained not by mean performance but by tail latency targets. Specifically, the 99th-percentile (p99) end-to-end latency often determines the required level of hardware overprovisioning, the feasibility of meeting user-facing SLOs, and ultimately the deployment cost.

A critical aspect of real-world LLM traffic that is often underrepresented in benchmarking studies is its heterogeneity. In practice, requests do not arrive with uniform prompt lengths. Short queries (a few tokens) and long document-grounded prompts (thousands of tokens) coexist in the same request stream. This heterogeneity creates an asymmetric interference pattern: long-prefill requests consume a disproportionate share of the batch token budget and GPU compute during the prefill phase, effectively blocking shorter requests from being scheduled promptly. The result is a form of

head-of-line blocking where short request tails are dragged upward by long-prefill contention, even when the serving system implements sophisticated batching and scheduling mechanisms.

In this paper, we present a systematic, reproducible performance study of the vLLM serving engine [Kwon et al., 2023, vLLM Team, 2026a] under controlled mixed prompt-length workloads. Our study makes the following contributions:

1. We construct controlled workload mixes at multiple short/long ratios and demonstrate that mixed prompt lengths cause nonlinear tail latency inflation: incorporating just 30% long prompts into a short-only stream increases the overall p99 by more than 50%.
2. We introduce a dynamic context budgeting mechanism that prevents token-limit overflow failures under variable prompt lengths, a practical reliability requirement for any mixed-length deployment.
3. We perform multi-dimensional sweeps over concurrency, per-batch token budget, and admission capacity, mapping the throughput-tail latency frontier and identifying the throughput saturation knee beyond which additional load yields no throughput gain but substantial tail degradation.
4. We conduct Pareto analysis over the two-dimensional configuration space (token budget $\times$ admission capacity) to provide actionable, SLO-aware tuning recommendations for mixed workloads.

The remainder of this paper is organized as follows. Section 2 covers relevant background on LLM serving systems and tail latency. Section 3 describes our experimental methodology, including workload construction, dynamic context budgeting, and the sweep protocol. Section 4 presents the results of five experiment groups. Section 5 discusses the implications for production deployment and the limitations of our study. Section 6 concludes.

2 Background and Related Work

2.1 Autoregressive LLM Inference

Transformer-based language models [Vaswani et al., 2017] generate text one token at a time. Each forward pass during the *decode* phase produces a single output token, and the cost of each decode step scales with the sequence length due to the key-value (KV) cache that must be maintained for attention computation. The *prefill* phase, which processes the entire input prompt in a single forward pass, is compute-intensive and its cost grows superlinearly with prompt length. This two-phase structure creates a fundamental asymmetry: a long prompt incurs a large upfront prefill cost, while subsequent decode steps are comparatively cheap on a per-token basis.

2.2 vLLM and Continuous Batching

vLLM [Kwon et al., 2023] is a widely deployed open-source serving engine that addresses several bottlenecks in LLM inference. Its core innovation, PagedAttention, manages the KV cache using a paging mechanism analogous to virtual memory, reducing internal fragmentation and enabling higher effective concurrency. vLLM also implements *continuous batching* [Yu et al., 2022], which dynamically admits new requests into an ongoing batch rather than waiting for all requests in the current batch to finish. This approach significantly improves GPU utilization compared to static batching.

More recent versions of vLLM (v1 and later) enable *chunked prefill* by default [Agrawal et al., 2024], a technique that splits long prefill operations into smaller chunks so that decode steps for other requests can be interleaved. While chunked prefill reduces the worst-case blocking from a single long prefill, it does not eliminate interference entirely, especially when configuration knobs permit large per-batch token budgets.

2.3 Tail Latency in Distributed and Serving Systems

The importance of tail latency in large-scale systems was articulated by Dean and Barroso [2013], who observed that in fan-out architectures, even rare slow responses can dominate the overall latency.

In the context of LLM serving, tail latency arises from multiple sources: queueing delays when the server is saturated, variable prefill costs across requests with different prompt lengths, KV cache pressure that forces preemption or swapping, and scheduling decisions that may inadvertently prioritize throughput over latency fairness.

2.4 Scheduling and Interference in LLM Serving

Several recent works address the tension between throughput and latency in LLM serving. Sarathi-Serve [Agrawal et al., 2024] introduced chunked prefill to tame the throughput-latency tradeoff by preventing long prefills from monopolizing entire batches. Splitwise [Patel et al., 2024] proposes phase splitting, where prefill and decode are handled by separate machine pools to avoid interference. Tao et al. [2025] propose prompt-aware scheduling strategies that account for prompt length when making admission and ordering decisions. SGLang [Zheng et al., 2024] introduces RadixAttention and other optimizations for structured LLM programs. Our work complements these efforts by providing an empirical characterization of the interference patterns that motivate such designs, using vLLM as a representative production serving engine.

3 Methodology

3.1 Serving Engine and Hardware

All experiments use vLLM v1 with its OpenAI-compatible API endpoint. We serve **Qwen2.5-3B-Instruct** [Qwen Team, 2025], a 3-billion parameter instruction-tuned model, on a single **NVIDIA L4 GPU** (24 GB VRAM) provisioned through Google Colab. The L4 represents a cost-effective inference-class accelerator commonly used in cloud deployments. Chunked prefill is enabled by default in vLLM v1 and is left active throughout all experiments unless otherwise noted.

3.2 Workload Construction

We construct controlled JSONL prompt sets using prompts sampled from two sources:

- **Short prompts** (8–24 tokens): sampled from Natural Questions [Kwiatkowski et al., 2019], representing concise factual queries.
- **Long prompts** (2,048–4,096 tokens): sampled from UltraChat [Ding et al., 2024], representing document-grounded or multi-turn conversational prompts.

We create four workload configurations by varying the short-to-long ratio:

1. **short_only**: 100% short prompts (control baseline).
2. **mix_90_10**: 90% short, 10% long.
3. **mix_70_30**: 70% short, 30% long.
4. **mix_50_50**: 50% short, 50% long (maximum interference stress test).

Requests are issued to the vLLM server using asynchronous HTTP clients at controlled concurrency levels. Each experiment run sends 200 or 400 requests (depending on the experiment), and we record the end-to-end latency, status code, prompt token count, and completion token count for every request.

3.3 Dynamic Context Budgeting

A practical challenge in mixed-length serving is that a fixed output token cap (e.g., `max_tokens=128`) can cause context-limit overflow when combined with long prompts, producing HTTP 400 errors. We implement a per-request dynamic context budgeting mechanism as follows:

The head-plus-tail trimming strategy preserves the instructional prefix (typically containing the system prompt and task description) and the most recent content at the end of the prompt, discarding only middle content. This design reflects the observation that in instruction-following settings, the beginning and end of a prompt carry the most task-relevant information.

Algorithm 1 Dynamic Context Budgeting

Require: Prompt p, desired max output tokens T_{desired}, model context limit L, safety margin δ, minimum output tokens $T_{\min}$

$n_p \leftarrow \text{EstimateTokens}(p)$ ▷ via chat template tokenization
$T_{\text{out}} \leftarrow \min(T_{\text{desired}},\ L - \delta - n_p)$
if $T_{\text{out}} < T_{\min}$ **then**
  $p \leftarrow \text{HeadTailTrim}(p,\ L - \delta - T_{\min})$ ▷ preserve prefix and suffix
  $n_p \leftarrow \text{EstimateTokens}(p)$
  $T_{\text{out}} \leftarrow L - \delta - n_p$
end if
assert $T_{\text{out}} > 0$
return (p, T_{out})

3.4 Server Configuration Knobs

We focus on two server-side configuration parameters that directly influence batching behavior and, consequently, the throughput-latency tradeoff:

- `max-num-batched-tokens` (**btok**): The maximum number of tokens the scheduler processes in a single iteration. A high value allows large prefill operations to fill the entire batch budget, increasing throughput but also increasing the blocking potential. A low value constrains each batch, limiting prefill dominance and potentially improving tail latency at the cost of throughput.
- `max-num-seqs` (**mns**): The maximum number of sequences the server admits concurrently. This controls the admission capacity: higher values allow more requests to be in-flight simultaneously, which can improve throughput via better GPU utilization but also increases queueing and contention at high load.

3.5 Configuration Regimes

We define two configuration regimes to isolate the effect of these knobs:

- **BAD (Unfair) Baseline**: `btok=8192`, `mns` left at a high default. This configuration deliberately encourages large prefill operations to dominate batches, reproducing the head-of-line blocking failure mode. It demonstrates that even with chunked prefill available, poor knob choices can undermine its benefits.
- **FAIR (Constrained) Configuration**: `btok` set to lower values (256–3072 in sweeps), limiting the per-batch prefill budget. This restricts how much prefill work any single scheduling iteration can perform, giving shorter requests more frequent scheduling opportunities.

3.6 Experimental Design

We organize the study into five experiment groups, each building on the findings of the previous one:

1. **Experiment 1: Mixed-workload baseline.** Fixed concurrency ($c = 8$), vary the prompt-length mix (short_only, 90/10, 70/30). Measures how the short/long ratio affects throughput and tail latency.
2. **Experiment 1.1: Dynamic context budgeting validation.** Repeats Experiment 1 with the dynamic budgeting mechanism enabled. Confirms that failures are eliminated without materially altering the latency profile.
3. **Experiment 3: Concurrency sweep under BAD config.** 50/50 workload, sweep concurrency from 1 to 64. Locates the throughput saturation knee and measures tail inflation beyond it.
4. **Experiment 4: 2D sweep (concurrency $\times$ admission).** 50/50 workload under BAD config, sweep concurrency $\in \{1, 2, 4, 8, 16, 32, 48, 64\}$ and `mns` $\in \{16, 32, 48, 64, 80, 96\}$. Tests whether admission capacity can compensate for the poor batching strategy.

5. **Experiment 5: Fair scheduling and Pareto analysis.** 50/50 workload at $c = 32$, sweep `btok` $\in \{256, \ldots, 8192\}$ and `mns` $\in \{32, \ldots, 128\}$. Maps Pareto frontiers of throughput versus short p99 and identifies SLO-optimal operating points.

3.7 Metrics

For each experimental run, we record and report:

- **Throughput**: requests per second (RPS), computed as successful requests divided by wall-clock time.
- **Failure rate**: fraction of requests returning non-200 status codes.
- **Overall latency percentiles**: p50, p90, and p99 of end-to-end request latency across all requests.
- **Stratified latency percentiles**: p50, p90, and p99 computed separately for short and long requests. This stratification is central to our analysis because it makes mixed-workload interference directly visible. A short p99 that approaches the long p99 is a clear indicator of head-of-line blocking.

4 Results

4.1 Experiment 1: Mixed-Workload Baseline

We begin by establishing how the short/long prompt ratio affects serving performance at a fixed concurrency of $c = 8$ with 200 requests per run. Table 1 summarizes the results.

Table 1: Experiment 1: Throughput and latency under varying prompt-length mixes ($c = 8$, 200 requests). Higher RPS is better; lower latency is better.

Workload	OK/Fail	RPS	p50 (s)	p90 (s)	p99 (s)	Mean (s)
short_only	200/0	2.53	3.55	3.58	3.59	3.10
mix_90_10	198/2	2.29	3.62	4.14	4.53	3.43
mix_70_30	195/5	1.96	4.10	4.89	5.38	4.05

Several patterns emerge from this baseline. Throughput degrades progressively as the fraction of long prompts increases: from 2.53 RPS for short-only down to 1.96 RPS for the 70/30 mix, a 22.5% reduction. The tail latency increase is even more pronounced. The overall p99 rises from 3.59 s to 5.38 s, an increase of approximately 50%. Notably, the penalty is nonlinear: the p99 increase from short-only to 90/10 is 0.94 s, while the increase from 90/10 to 70/30 is 0.85 s, despite the latter representing a smaller incremental change in mix ratio. This superlinear behavior reflects the compounding effect of long-prefill contention in the scheduling queue.

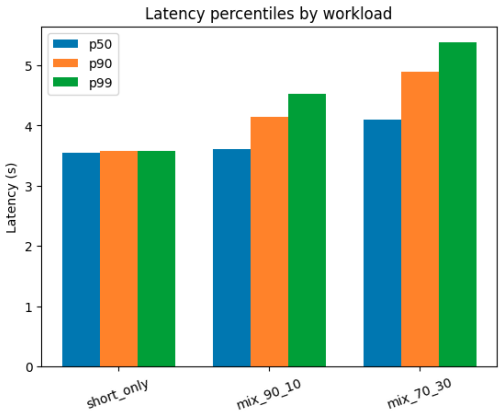
A critical observation is the emergence of hard failures. With a fixed `max_tokens=128`, 2 out of 200 requests fail in the 90/10 mix and 5 fail in the 70/30 mix, all due to context-limit overflow (HTTP 400). This demonstrates that naive token allocation is not viable under variable prompt lengths.

Figure 1a illustrates the widening gap between p50 and p99 as the long-prompt fraction grows, confirming that the tail is disproportionately affected. The latency histogram for the 70/30 mix (Figure 2) reveals a distinctly bimodal distribution, with a cluster of fast short-prompt responses and a heavier right tail from long-prompt interference.

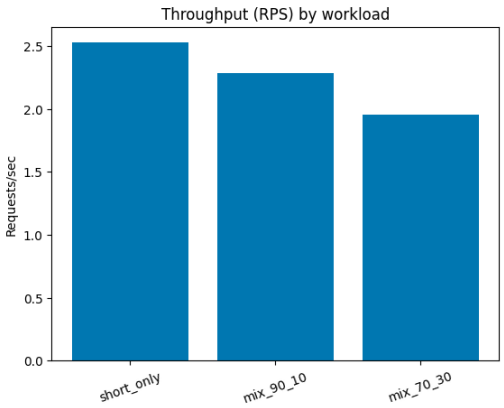
4.2 Experiment 1.1: Dynamic Context Budgeting

We repeat the baseline experiment with the dynamic context budgeting mechanism (Algorithm 1) enabled. Table 2 presents the results.

The results confirm two properties of the budgeting mechanism. First, all 200 requests succeed across every workload configuration, eliminating the HTTP 400 failures observed in Experiment 1. Second, the latency and throughput profiles are virtually identical to the non-budgeted runs (RPS within



(a) Latency percentiles (p50, p90, p99) across workloads.



(b) Throughput (RPS) across workloads.

Figure 1: Experiment 1: Baseline measurements showing progressive degradation of both throughput and tail latency as the proportion of long prompts increases.

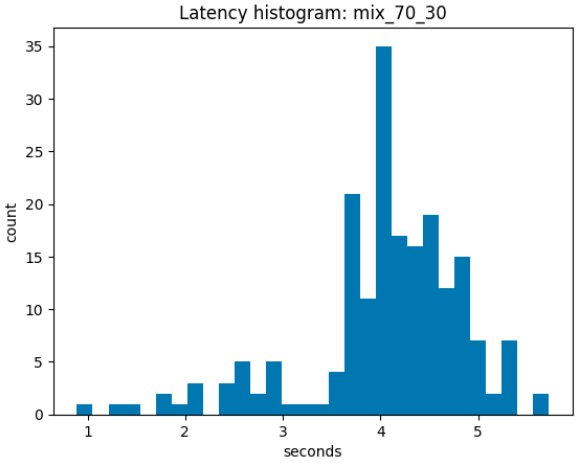


Figure 2: Latency histogram for the 70/30 mixed workload, showing bimodal structure from short/long request interference.

1%, p99 within 0.01 s), confirming that the budgeting logic does not introduce measurable overhead or alter the workload's computational characteristics. Dynamic context budgeting is therefore a necessary reliability measure that can be deployed transparently.

4.3 Experiment 3: Concurrency Sweep Under BAD Configuration

Having established the baseline interference pattern, we now examine how concurrency interacts with the deliberately poor (BAD) server configuration. We fix the workload at 50/50 and sweep concurrency from 1 to 64. Figure 3 and Table 3 summarize the results.

The throughput curve exhibits a clear saturation knee at $c \approx 16$, where RPS reaches approximately 5.9 and then plateaus through $c = 64$. Beyond this knee, additional concurrency does not produce meaningful throughput gains. The tail latency, however, continues to grow aggressively: the overall p99 increases from 6.20 s at $c = 16$ to 21.67 s at $c = 64$, a factor of 3.5×. This demonstrates that throughput saturation does *not* imply latency safety.

The stratified p99 view (Figure 3b) reveals the mixed-workload interference mechanism in detail. At $c = 1$, short requests complete in 0.46 s (p99) while long requests take 1.30 s, a natural gap reflecting the difference in prefill cost. As concurrency increases, the short p99 inflates rapidly and

Table 2: Experiment 1.1: Results with dynamic context budgeting enabled ($c = 8$, 200 requests).

Workload	OK/Fail	RPS	p50 (s)	p90 (s)	p99 (s)	Mean (s)
short_only	200/0	2.52	3.56	3.57	3.58	3.11
mix_90_10	200/0	2.27	3.64	4.24	4.53	3.47
mix_70_30	200/0	1.94	4.18	4.89	5.39	4.07

Table 3: Experiment 3: Concurrency sweep under BAD configuration (50/50 mix, 200 requests per point). Short and long p99 are stratified by prompt class.

c	RPS	p50 (s)	p99 (s)	Short p99 (s)	Long p99 (s)	Mean (s)
1	1.31	0.46	1.30	0.46	1.30	0.76
2	2.35	0.75	1.79	0.98	1.80	0.85
4	3.59	0.97	2.69	1.87	2.71	1.10
8	4.90	1.47	3.69	2.76	3.72	1.62
16	5.90	2.54	6.20	4.03	6.21	2.65
32	5.91	5.83	9.54	7.24	10.19	5.36
48	6.03	7.49	13.76	11.28	13.87	7.45
64	5.82	9.91	21.67	15.79	21.94	10.59

converges toward the long p99: by $c = 64$, short p99 has reached 15.79 s, a 34× increase from its solo value. Short requests, which should complete quickly, are being held in the scheduling queue behind long-prefill work.

Figure 4a visualizes the throughput-tail tradeoff directly: the curve bends sharply at the knee, indicating that any concurrency beyond $c \approx 16$ purchases negligible RPS at substantial tail cost. The tail inflation ratio (Figure 4b) quantifies the distributional change: the p99/p50 ratio grows from roughly 2.8× at low concurrency to over 2.2× at high concurrency, indicating that the latency distribution becomes increasingly heavy-tailed under load.

4.4 Experiment 4: Two-Dimensional Sweep (Concurrency × Admission Capacity)

We next investigate whether the admission capacity knob (`max-num-seqs`) can mitigate the tail inflation observed under the BAD configuration. We sweep concurrency $\in \{1, 2, 4, 8, 16, 32, 48, 64\}$ and `mns` $\in \{16, 32, 48, 64, 80, 96\}$ for a total of 48 configurations, each running 200 requests on the 50/50 workload.

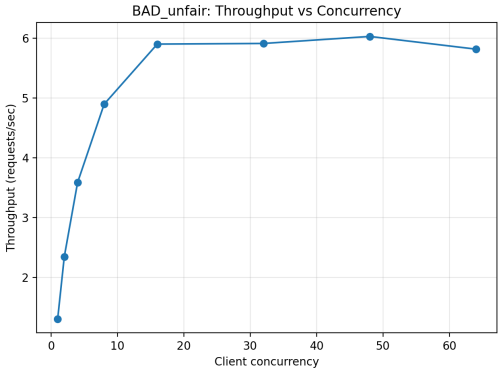
The RPS heatmap (Figure 5a) confirms that higher admission capacity unlocks more throughput at high concurrency. At $c = 64$, increasing `mns` from 16 to 64 raises RPS from approximately 13.6 to 20.2, a 49% improvement. However, the short p99 heatmap (Figure 5b) tells a different story. While lower admission values (e.g., `mns`=16) produce moderate absolute tails due to their conservative admission policy, higher admission values (e.g., `mns`=96) exhibit irregular tail spikes from increased contention and queueing pressure.

The long p99 heatmap (Figure 6) shows that long-request tails are relatively insensitive to the admission capacity setting but grow monotonically with concurrency. This asymmetry confirms that the admission capacity knob primarily affects how many requests are in-flight (and thus throughput), but does not address the fundamental batching-level interference between short and long prefills. Under the BAD configuration, the per-batch token budget remains high regardless of `mns`, so long prefills continue to dominate individual scheduling iterations.

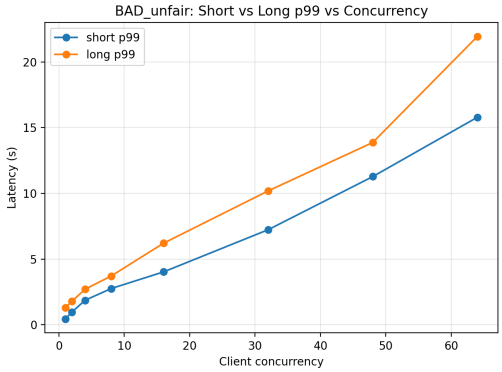
The key conclusion from Experiment 4 is that `max-num-seqs` functions as a throughput lever, not a tail latency fix. To address interference at the batch level, a different knob is required.

4.5 Experiment 5: Fair Scheduling and Pareto Analysis

Motivated by the finding that admission capacity alone cannot control tail latency, we turn to the per-batch token budget (`max-num-batched-tokens`) as a mechanism for fair scheduling. The core

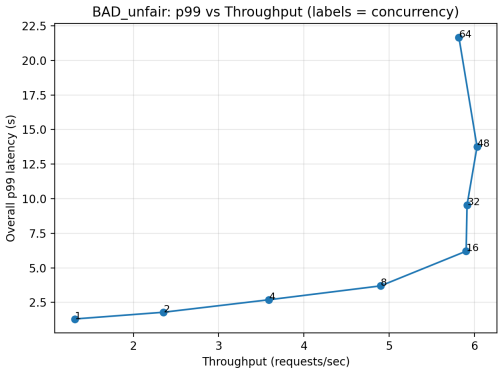


(a) Throughput (RPS) vs. concurrency.

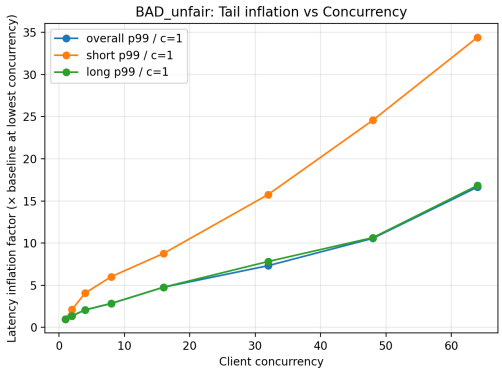


(b) Stratified p99: short vs. long requests.

Figure 3: Experiment 3: Throughput saturates around $c = 16$ while tail latency continues to grow sharply. Under the BAD configuration, short request p99 is dragged toward the long request p99, indicating severe head-of-line blocking.



(a) Overall p99 vs. RPS tradeoff curve.



(b) Tail inflation ratio (p99/p50) vs. concurrency.

Figure 4: Experiment 3 (continued): The p99 vs. RPS curve (left) shows diminishing throughput returns for increasing tail cost. The tail inflation ratio (right) captures how the distribution shape changes under load.

idea is to constrain how much prefill work any single scheduling iteration can perform, thereby limiting the ability of long prompts to monopolize the batch.

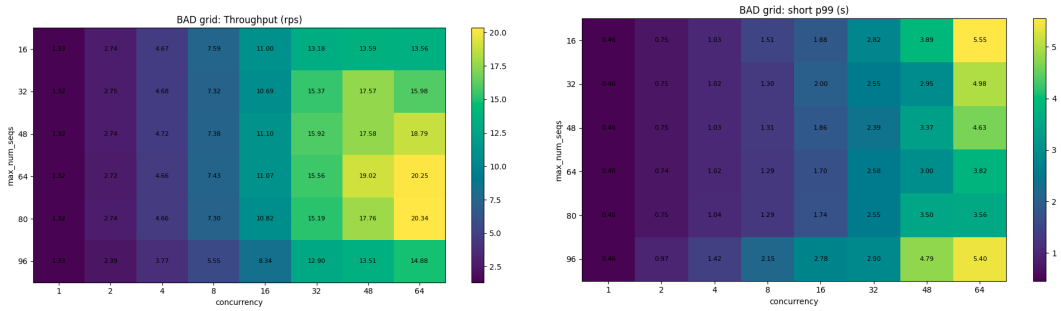
4.5.1 Sanity Check: FAIR vs. BAD at $c = 32$

We first validate that reducing `btok` has a measurable effect. At $c = 32$ with `btok`=768 (FAIR) versus `btok`=8192 (BAD), we observe that the FAIR configuration reshapes the latency distribution: p90 improves (9.18 s vs. 10.46 s for BAD) and short p99 marginally improves (10.08 s vs. 10.15 s). The p50, however, worsens slightly (6.04 s vs. 5.53 s), reflecting the throughput cost of smaller batches. This confirms that `btok` is a functional lever but that its optimal value is not obvious.

4.5.2 1D Sweep: Token Budget at $c = 16$ and $c = 32$

At $c = 16$, increasing `btok` from 256 to 2048 improves the overall p99 from 7.49 s to 5.88 s (a 21.5% reduction), p50 from 3.92 s to 2.88 s, and long p99 from 7.64 s to 5.97 s. In this regime, the server is prefill-throttled at low token budgets: restricting `btok` too aggressively starves the scheduler and increases queueing.

At $c = 32$, the picture changes. RPS increases 38.1% as `btok` grows from 256 to 3072 (3.97 to 5.48 RPS). But the p99 behavior is non-monotonic: the best overall p99 occurs at mid-range token budgets, and tails worsen at both extremes. At very low `btok`, scheduler starvation increases queueing delay.



(a) RPS heatmap.

(b) Short p99 heatmap.

Figure 5: Experiment 4: Heatmaps of RPS (left) and short p99 (right) over the concurrency $\times$ `max-num-seqs` grid under the BAD configuration. Higher admission capacity increases throughput but does not systematically reduce tails.

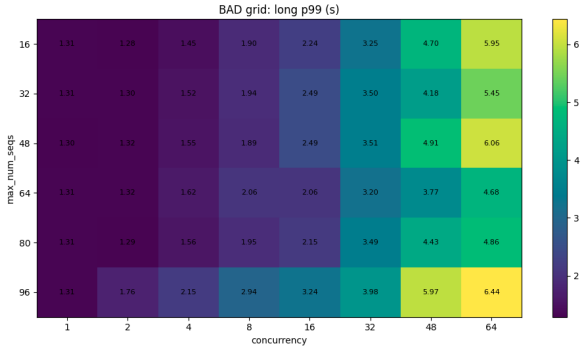


Figure 6: Experiment 4: Long p99 heatmap. Long-request tails are less sensitive to `max-num-seqs` variation but grow monotonically with concurrency.

At very high `btok`, long prefills regain the ability to dominate batches. The sweet spot depends on both the concurrency level and the workload composition.

4.5.3 2D Grid Sweep: Token Budget $\times$ Admission Capacity

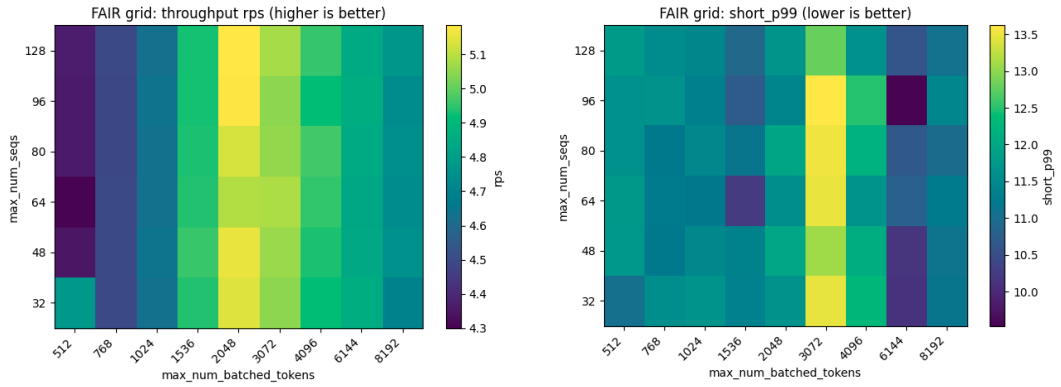
We perform a comprehensive 2D sweep at $c = 32$, varying `btok` across nine levels (512, 768, 1024, 1536, 2048, 3072, 4096, 6144, and 8192) and `mns` across four levels (32, 64, 96, and 128), producing 36 configurations in total, with additional points from the BAD grid included for comparison.

The FAIR grid (Figure 7) reveals that throughput varies modestly across the configuration space (approximately 4.3 to 5.2 RPS), but the short p99 varies significantly, from under 10 s to over 12 s depending on the combination. The best short-tail protection we observe is at `btok`=6144 and `mns`=96, which yields a short p99 of approximately 9.53 s compared to 10.15 s under the BAD baseline (a 6.1% improvement). However, this configuration increases the long p99 to approximately 13.09 s, reflecting the fundamental tension between optimizing for different request classes.

4.5.4 Pareto Analysis

The most informative view of the configuration space is the Pareto frontier, which identifies configurations where one cannot improve throughput without worsening short p99, or vice versa.

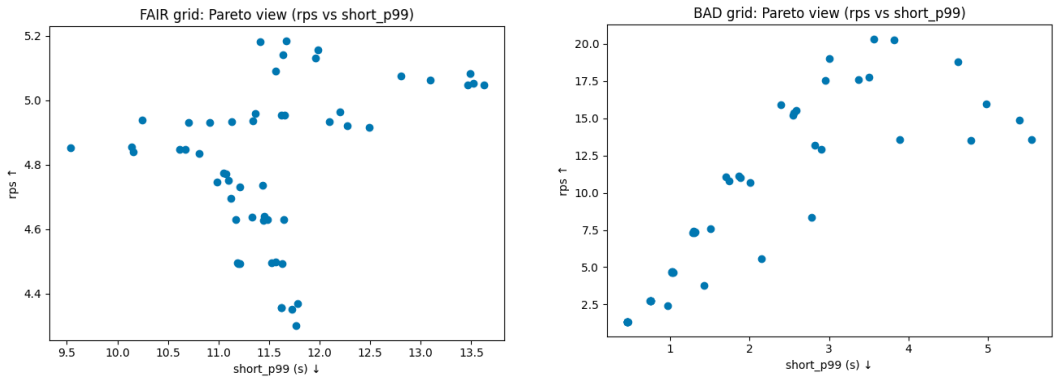
Figure 8 shows the Pareto frontiers for both the FAIR and BAD configurations. The FAIR frontier is notably tighter, with less variance in short p99 across configurations that achieve similar throughput. The BAD frontier, by contrast, exhibits wider scatter, reflecting the higher sensitivity of the BAD configuration to parameter choices.



(a) FAIR grid: RPS heatmap.

(b) FAIR grid: short p99 heatmap.

Figure 7: Experiment 5: FAIR configuration heatmaps at $c = 32$. Throughput varies modestly across the grid (4.3–5.2 RPS), but short p99 varies substantially depending on the (`btok`, `mns`) combination.



(a) FAIR Pareto frontier: RPS vs. short p99.

(b) BAD Pareto frontier: RPS vs. short p99.

Figure 8: Pareto frontiers for the FAIR (left) and BAD (right) configurations. Each point represents a (`btok`, `mns`) combination. Points on the frontier represent efficient operating points where neither metric can be improved without degrading the other.

Figure 9 directly compares the two strategies by plotting RPS against the change in short p99 relative to the BAD baseline. The conclusion is that there is no single "best" configuration. The optimal operating point depends entirely on the deployment's SLO:

- To minimize **short p99**: select higher `btok` with moderate `mns` (e.g., `btok`=6144, `mns`=96).
- To minimize **long p99**: select conservative values for both knobs (e.g., `btok`=512, `mns`=32).
- For a **balanced operating point**: select the knee of the Pareto frontier, then validate with a targeted concurrency sweep.

5 Discussion

5.1 Summary of Findings

Our experiments yield five principal findings that carry implications for the operation of LLM serving systems in production:

Finding 1: Nonlinear tail inflation from mixed prompts. Incorporating long prompts into a short-only workload creates disproportionate tail latency inflation. With just 30% long prompts, the overall p99 increases by over 50% (from 3.59 s to 5.38 s). The relationship between the long-prompt

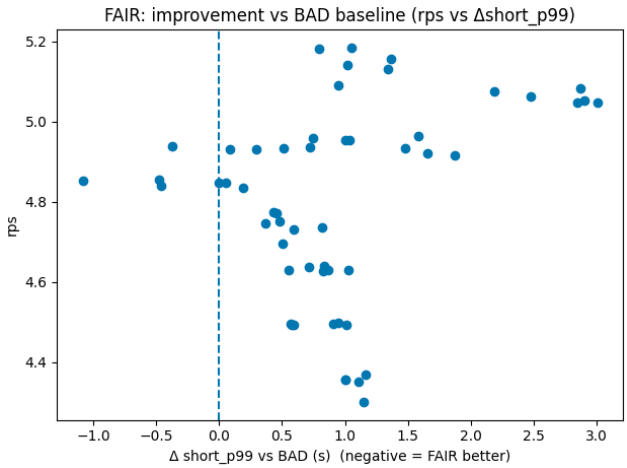


Figure 9: FAIR vs. BAD comparison: RPS vs. the change in short p99 relative to the BAD baseline. Points below the $y = 0$ line represent configurations where the FAIR approach reduces short p99 relative to BAD.

fraction and tail inflation is superlinear, suggesting that even modest heterogeneity in prompt length can have outsized effects on worst-case latency.

Finding 2: Dynamic context budgeting is a prerequisite. Without per-request token budget management, mixed-length workloads produce hard failures (HTTP 400 context-limit overflow). Our dynamic budgeting mechanism eliminates these failures with negligible overhead (less than 1% change in throughput or latency). This is not an optimization but a correctness requirement for any deployment handling variable-length prompts.

Finding 3: Throughput saturation does not imply latency safety. The throughput curve under the BAD configuration saturates at $c \approx 16$ (approximately 5.9 RPS), but the p99 latency continues to grow by $3.5\times$ from $c = 16$ to $c = 64$. This is a dangerous operating regime: monitoring throughput alone would suggest the system is performing well, while tail latency is degrading severely. Operators must monitor tail metrics, not just throughput, to detect this failure mode.

Finding 4: Admission capacity is not a substitute for batching control. Increasing `max-num-seqs` improves throughput by up to 49% at high concurrency, yet it does not systematically reduce tail latency. The admission capacity determines how many requests are in-flight concurrently; it does not govern how the scheduler distributes compute within each batch. As a result, under a high per-batch token budget, long prefills continue to dominate individual scheduling iterations regardless of how permissive the admission setting is.

Finding 5: Constrained token budgets enable SLO-aware tuning. The per-batch token budget (`max-num-batched-tokens`) is the effective lever for controlling mixed-workload interference at the batch level. Combined with Pareto analysis over the 2D configuration space, it enables deployment teams to select operating points that respect their specific SLO constraints, whether those prioritize short-request responsiveness, long-request completion, or a balanced tradeoff.

5.2 Practical Recommendations

Based on our findings, we offer the following recommendations for practitioners deploying vLLM (or similar continuous-batching serving engines) under mixed prompt-length workloads:

1. **Always implement dynamic context budgeting.** Variable prompt lengths require per-request token allocation to prevent hard failures. The output-first, input-second trimming strategy is both simple and effective.

2. **Identify the throughput saturation knee.** Run a concurrency sweep at the target workload mix and locate the point where throughput gains plateau. Operating beyond this point wastes tail latency budget for negligible throughput return.
3. **Use `max-num-batched-tokens` as the primary tail-control knob.** Rather than relying on admission capacity alone, constrain the per-batch token budget to limit prefill dominance. The optimal value is workload-dependent and should be determined empirically.
4. **Sweep the 2D configuration space and compute Pareto frontiers.** The interaction between `btok` and `mns` is non-trivial. A grid sweep followed by Pareto analysis provides the most complete picture of the throughput-tail tradeoff for a given workload.
5. **Monitor stratified tail metrics.** Overall p99 can mask class-specific interference. Reporting short p99 and long p99 separately makes it possible to detect head-of-line blocking and to evaluate whether a configuration change improves fairness.

5.3 Limitations

Several limitations of this study should be noted. First, all experiments are conducted on a single GPU (NVIDIA L4) with a single model (Qwen2.5-3B-Instruct). Different hardware (e.g., A100, H100) and larger models may exhibit different saturation points and interference patterns due to differences in compute-to-memory ratios and KV cache capacity. Second, our workloads use synthetic prompt mixes with controlled length distributions; real production traffic may exhibit more complex patterns including temporal burstiness, correlated prompt lengths, and varying output-length requirements. Third, we do not explore all available vLLM knobs (e.g., preemption policy, swap space, speculative decoding), each of which may interact with the knobs we study. Fourth, while our Pareto analysis identifies efficient operating points, the frontiers may shift under different workload compositions, requiring re-characterization when the traffic mix changes. Finally, our measurements capture end-to-end latency but do not decompose it into queueing delay, prefill time, and decode time; integrating vLLM's Prometheus metrics [vLLM Team, 2026b] would enable deeper root-cause analysis and constitutes a natural extension.

6 Conclusion

We have presented a systematic empirical study of tail latency and throughput behavior in the vLLM serving engine under mixed prompt-length workloads. Our experiments demonstrate that prompt-length heterogeneity creates substantial and nonlinear tail latency inflation, that throughput saturation is a misleading indicator of system health, and that the per-batch token budget is the most effective knob for controlling mixed-workload interference at the scheduling level. Through Pareto analysis over the two-dimensional configuration space of token budget and admission capacity, we provide a principled framework for SLO-aware operating-point selection.

All experimental infrastructure, workload files, sweep runners, and analysis code are released as a self-contained, reproducible benchmarking lab notebook. We believe this lab can serve as a reusable template for evaluating serving-system performance under realistic, heterogeneous workloads, and we hope it contributes to more SLO-aware evaluation practices in the LLM serving community.

Acknowledgments and Disclosure of Funding

This work was conducted as part of the course project for DSC 291: Efficient AI for Data Science at the University of California, San Diego (Winter 2026). We thank the course instructor for valuable discussions on efficient inference and serving systems. Experiments were run on Google Colab with NVIDIA L4 GPU resources.

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations, 2024.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165. arXiv:2309.06180.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

Yuxin Tao, Yibo Zhang, Matthew T. Dearing, Xingzhi Wang, Yufei Fan, and Zhiling Lan. Prompt-aware scheduling for low-latency LLM serving. *arXiv preprint arXiv:2510.03243*, 2025.

Ashwin Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

vLLM Team. vLLM documentation: Overview and serving features. `https://docs.vllm.ai/`, 2026a. Accessed March 2026.

vLLM Team. vLLM metrics design and production metrics. `https://docs.vllm.ai/en/stable/design/metrics/`, 2026b. Accessed March 2026.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. ORCA: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kober, Ke Shi, Joseph E. Gonzalez, Ion Stoica, Hao Zhang, and Tianqi Chen. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.